CS1810 Oct. 27 Notes

Non-Deterministic Finite Automata

Equivalence of Machines: Machine M_1 is equivalent to Machine M_2 if $L(M_1) = L(M_2)$

Regular Expressions

Regular expressions describe regular languages.

Example: $(a + b \cdot c)^*$ describes the language $\{a, bc\}^* = \{\lambda, a, bc, aa, abc, bca, \ldots\}$

Recursive definition:

primitive regular expressions: $\emptyset$, λ, a

Given regular expressions r_1 and r_2

$r_1 + r_2$
$r_1 \cdot r_2$
r_1^*
(r_1)

} Are regular expressions

A regular expression: $(a+b \cdot c)^* \cdot (c + \emptyset)$

Not a regular expression: $(a + b+)$

Languages of Regular Expressions

$L(r)$: language of regular expression r

Example:

$L((a+b \cdot c)^*) = \{\lambda, a, bc, aa, abc, bca, \ldots\}$

For primitive expressions

$L(\emptyset) = \emptyset$

$L(\lambda) = \{\lambda\}$

$L(a) = \{a\}$

For regular expressions r_1 and r_2

$L(r_1 + r_2) = L(r_1) \cup L(r_2)$

$L(r_1 \cdot r_2) = L(r_1) L(r_2)$

$L(r_1^*) = (L(r_1))^*$

$L((r_1)) = L(r_1)$

Some examples:

regular expression $r = (aa)^*(bb)^* b$

$L(r) = \{a^{2n} b^{2m} b : n, m \geq 0\}$

regular expression $r = (0+1)^* 00 (0+1)^*$

$L(r) = \{$ all strings containing substring $00\}$

Equivalent regular expressions: Regular expressions r_1 and r_2 are equivalent if $L(r_1) = L(r_2)$

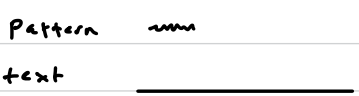
Theorem:

{Languages generated by regular expressions} = {regular languages}

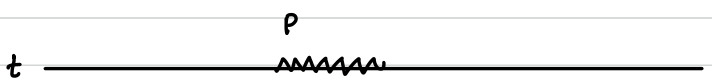
Fundamental Pattern Matching Problem

Pattern $P = p_1 p_2 \dots p_\ell$, $p_i \in \Sigma$

Text $t = t_1 t_2 \dots t_m$, $t_j \in \Sigma$



Find first exact match of pattern in text



The Knuth - Morris - Pratt Algorithm (KMP - algorithm)

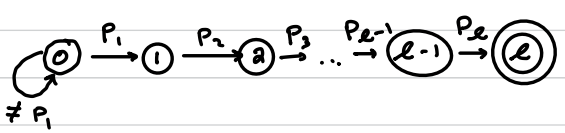


$P = p_1 p_2 \dots p_\ell$

$t = t_1 t_2 \dots t_m$

We first construct a pattern matching machine M_P

We start by constructing a skeletal DFA machine.



We think of state i as a pointer to the ith position in pattern P.

M_P is like a DFA except some part of it

We will extend the skeletal machine to a DFA M_P

The state j of M_P corresponds to the prefix $P_1 P_2 \dots P_j$ of the pattern P.

M_P starts in state 0 with its reading head reading first symbol of the input tape t_1.

IF $t_1 = P_1$, then M_P enters state 1 and advances its reading head to position 2 on the input tape, namely to t_2.

IF $t_1 \neq P_1$, then M_P remains in state 0 and advances its reading head to position 2 on the text/input tape, i.e. t_2.

Suppose after having "read" $t_1 t_2 \dots t_K$, we have M_P is in state j. This implies that the last j symbols of $t_1 t_2 \dots t_K$ are $P_1 P_2 \dots P_j$, and the last m symbols of $t_1 t_2 \dots t_K$ are not a prefix of $P_1 P_2 \dots P_j$. $m > j$

IF $t_{K+1} = P_{j+1}$, then M_p enters state $j+1$ and advances its head to t_{K+2}

IF $t_{K+1} \neq P_{j+1}$, then M_p enters the highest number state i such that $P_1 P_2 \ldots P_i$ is a suffix of $t_1 t_2 \ldots t_{K+1}$

To determine i, M_p uses a failure function algorithm

- f is the failure function: $f(j)$ is the largest s less than j for which $P_1 P_2 \ldots P_s$ is a suffix of $P_1 P_2 \ldots P_j$